

BiasLens: A Graph ML Approach for Detecting Bias in News

Abstract. This paper addresses the challenges posed by pervasive bias within political news sources. Many bias detection tasks primarily focus on the textual content of articles and neglect the underlying influence of authors and news outlets. To counteract this issue, we propose constructing a Knowledge Graph (KG) and using extracted embeddings to classify the bias of an article. We created a KG that encapsulates the rich information and intricate relationships among news articles, authors, and news outlets. Using a pre-labeled news dataset, we constructed the KG, employed a walking strategy to visit nodes, and extracted embedding vector representations for each article node. We then divided the embeddings into sets for training and testing a Machine Learning (ML) model. The study demonstrated the effectiveness of our strategy in classifying bias in news articles, achieving an accuracy of $\sim 96\%$, as opposed to more traditional techniques such as Artificial Neural Networks (ANNs) with an accuracy of $\sim 38\%$. Our approach offers a novel perspective on News Bias Classification, showing the potential of using KGs in ML tasks. This method could serve as a foundation for future advancements in bias detection and analysis in the media industry.

Keywords. Bias Detection, Knowledge Graph, News Classification

1. Introduction

The role of news media in shaping public opinion and influencing societal discourse is undeniable. Readers rely on news outlets to provide accurate, unbiased information, especially when it comes to controversial issues. However, the reality is that bias in news reporting is prevalent, with many news sources exhibiting a tendency to favor particular perspectives or ideologies [6]. This bias can have far-reaching consequences, impacting the way people perceive and interpret current events.

Combating news bias requires firstly identifying those biases. News aggregators like All-Sides¹ and Ground News² have emerged, as solutions for readers to access news stories from various viewpoints. The underlying premise is to present the user with the articles taken from multiple sources known for their varying political leaning about a particular news event, this may feel tedious and overwhelming to casual news consumers.

Despite the efforts of news aggregators, bias in news reporting remains a complex and challenging problem. Traditional methods of detecting bias, such as manual fact-

¹<https://www.allsides.com/unbiased-balanced-news>

²<https://ground.news/>

checking and editorial oversight, are time-consuming and resource-intensive. Natural Language Processing (NLP) and Machine Learning (ML) are the go-to solutions for automating this task, but they still come with their own issues as they are limited to what is mentioned in the article text and cannot make meaningful connections with external information.

Experienced readers will often take into account details about a news article such as the author and the publishing outlet both of which may exert influence over a news article, and many may have a history of putting out biased news. These factors are often ignored when modeling bias detection as an NLP problem thus we propose BiasLens, a novel approach to bias detection in news articles using Knowledge Graphs (KGs). By representing news articles and their in a KG, we can capture the complex relationships and contextual information that contribute to bias in news reporting. This approach has the potential to be of value for indicating where an article lives on a bias rating spectrum, offering the sorely needed insight that news consumer needs about their daily news.

2. Related Works

During our extensive literature review, we came across many examples of KGs being used in various recommendation and detection tasks similar to our approach, though to the best of our knowledge, using KGs to detect news bias is a novel idea but the use of KGs in the news domain, in general, is not new.

Bias detection has evolved to become more nuanced and context-aware, [10] proposed Bias-Aware Neighborhood Contexts (BANC), this approach labels bias on a sentence level but considers the bias label of neighboring sentences and adds them to the input of the model. They tested their approach with models such as BERT [5] and RoBERTa [9] and found that BANC method significantly improved F1-scores 58.15 with BERT, a notable improvement over the regular setup which had an F1-score of 42.32.

[2] proposed a KG approach for informing Fake News Detection using LLMs. Their methodology extracts graph embeddings from the WikiData5M KG based on the most relevant triplets in an article text. Combining these embeddings with the vector obtained from embedding the article corpus using LLMs resulted in a Knowledge-enriched document vector. Their enhanced document vectors outperformed their baseline with a maximum accuracy of 97.06% being highly competitive with the state-of-the-art in Fake News detection.

[8] discussed using KGs to enhance a news recommendation algorithm since the time-sensitive nature of news means that interaction data may be sparse and not fully representative of a user's use. Their solution to this problem was building a news-specific KG to fill the data sparsity of interaction data to recommend news more accurately, they managed to outperform other methods with an accuracy of 0.8503 and F1-score of 0.7992 in local news detection.

Another task is fake news detection. [11] discussed the spread of news on social media and proposed DEAP-FAKED a dual-part KG-based framework to solve the problem. Their approach combines NLP methods to encode news headlines and embedded KG entities. DEAP-FAKED achieved an F1-score of 88% and 78% on two public datasets which outperformed other classification methods.

In their work, [7] developed CompareNet, a graph neural model that differentiates between satirical and trusted news articles. CompareNet outperformed the state-of-the-art models, attaining 3% improvements in both micro and macro F1 scores. Their model benefited from topic information and connections to an external knowledge base (KB), but removing topic information led to significant performance decreases of 4.0-5.0% in F1 scores. This emphasizes the need to construct a robust and descriptive KG to fulfill the goal of our product.

3. Dataset

In this section, we will discuss the dataset used in this study. The dataset by [1] consists of 37,737 articles, collected from AllSides, each article is labeled with a political bias rating of either left, center, or right. It is important that the authors mention the center means a centrist political view rather than the lack of a bias or "no bias". This dataset stands out as AllSides employs a systematic process with various steps including a Blind Bias Survey³ where individuals with various political ideologies read headlines and provide a bias rating without knowing the source the article comes from which both saves time and eliminates bias from our end if we were to label the dataset from our end.

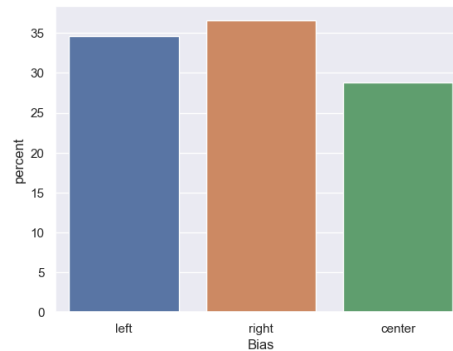


Figure 1. T-Box Definition

The dataset included detailed descriptions of each article including Title, Content, Topic, Source, Creator, and Bias. Using regular expressions (regex), we performed text cleaning on creator names since multiple authors can be associated with a single article separated by commas. We split the names and removed special characters and double spaces to make sure they are consistent across records. For missing names we opted to replace them with "Anonymous" in order to preserve completeness and consistency

³<https://www.allsides.com/blind-bias-survey>

when it comes time to construct the graph. In addition we will stick with the three bias classes included in the dataset, thus we observed the class distribution to make sure no class is over-represented. We do not intend to extract any features from the content of the article thus we did not process their text.

4. KG Construction

In this section we will dive deeper into the graph structure and its construction. The KG construction was highly dependent on the structure of the dataset. Our methodology for constructing the KG can be split into 2 parts, The T-Box and A-Box.

T-Box From the available data we chose to define the following three classes: Article, Author, and Outlet. The classes can then be linked using the following relationships, "writtenBy" links an article with an author, and "publishedBy" links an article with an outlet. We choose these classes and relationships with future expansion in mind and included data properties for the article class to include, body, id, and most importantly, bias. T-Box structure can be seen in Figure 2.

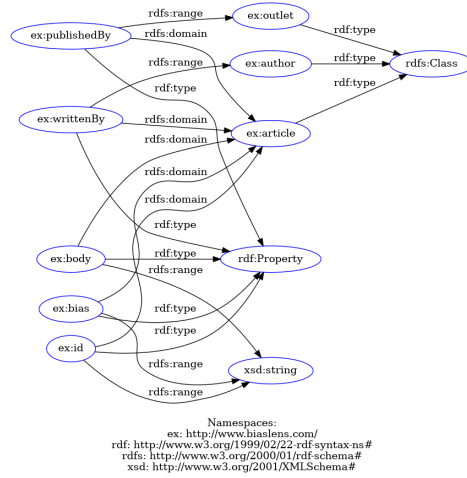


Figure 2. T-Box Definition

A-Box Using rdflib [4] python package we iterated over the records in our dataset and mapped the individual records to instances of the article, author, and outlet classes. For each record, we created an Article instance with properties body, id, and bias filled with the corresponding data from the record. We also created Author and Outlet instances based on the author and outlet information in the record. Then, we used the writtenBy and publishedBy relationships to link the Article instance with the corresponding Author and Outlet instances. We will serialize our graph into a turtle file.

5. Machine Learning Algorithms

This is ultimately a node classification task for each news article node in the KG, in our case, we have three classes as articles in our dataset can be either labeled as Left, Center, or Right. We will experiment with a handful of ML algorithms on the constructed KG, in addition we need to establish a basic baseline by attempting to classify articles based purely on their textual content, this part is important for verifying our hypothesis around a graph based approach.

5.1. Baseline

In our baseline approach, our focus was on establishing a strong foundation for detecting bias by applying NLP text preprocessing techniques to the text body of the article. The preprocessing phase involved the elimination of stop words and punctuation followed by lemmatization. We moved on to extracting features from the text by computing the TF-IDF matrix for all articles leaving us with a feature vector for each article. We proceeded to split vectors into testing and training sets for use with a handful of ML algorithms. Table 1 summarizes the results of this task.

Table 1. Baseline accuracy scores in text classification.

Machine Learning Model	Accuracy
Logistic Regression	0.383
Random Forest	0.447
SVC	0.402
MLP	0.384

5.2. Using Graph Embeddings

As each article is represented as a node in our KG it will be possible to a vector representing each node in the graph, this can be done by performing a set of graph walks starting at each article node employing either a Breadth-First-Search (BFS) or Depth-First-Search (DFS) or a mix of both DFS and BFS. How we choose to traverse the graph will be called the walk strategy. In this approach we aim to extract node embeddings for each article in our KG for use in training and testing ML models, this can be done using pyRDF2Vec [14], which is a python implementation of the RDF2Vec algorithm proposed in [13], using a defined walk strategy over the graph and using these walks as input for the word2vec algorithm [12] to extract embedding representation of our article nodes. For an example graph like the one in Figure 3, a set of walks for an article could look like [Article_1, Author_A] [Article_1, Outlet_XYZ]

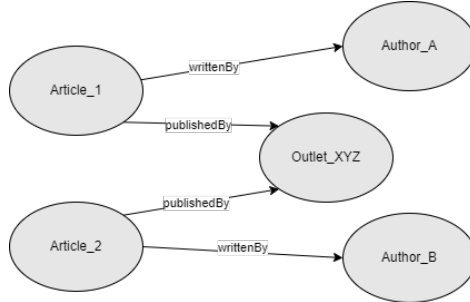


Figure 3. Example Graph

One of the key benefits of using pyRDF2Vec is inclusion of a suite of walk strategies, many of which were proposed in [16], so as part of our experiment design, Table 2 contains all the graph walk strategies and their descriptions as per pyRDF2Vec documentation while Tables 3 and 4 show all the walk strategies employed and the different ML algorithms paired with them. To preserve consistency across runs we fixed the word2vec algorithm to train for 20 epochs and set a max walk depth of 2, all runs used a 20/80 test-train split.

Table 2. Summary of pyRDF2Vec walk strategies

Walker	Description
HALKWalker	HALK walking strategy which removes rare vertices from randomly extracted walks, increasing the quality of the generated embeddings while memory usage decreases.
NGramWalker	N-Gram walking strategy which relabels the n-grams in random walks to define a mapping from one-to-many. The intuition behind this is that the predecessors of a node that two different walks have in common can be different.
RandomWalker	Random walking strategy which extracts walks from a root node using the Depth First Search (DFS) algorithm if a maximum number of walks is specified, otherwise the Breadth First Search (BFS) algorithm is used.
WalkletWalker	Walklets walking strategy which transforms randomly extracted walks into walklets which are walks of size one or two including the root node and potentially another vertex that can be a predicate or an object.

Table 3. Performance Metrics of Different Walking Strategies with Various ML Models (Accuracy)

Model	HALKWalker	NGramWalker	RandomWalker	WalkletWalker
XGBClassifier	0.9583	0.9182	0.9583	0.9289
LogisticRegression	0.9176	0.8482	0.9296	0.8864
RandomForest	0.9450	0.8941	0.9478	0.9225
SVC	0.9406	0.8951	0.9449	0.9057
KNeighborsClassifier	0.9587	0.9027	0.9607	0.9189

Table 4. Performance Metrics of Different Walking Strategies with Various ML Models (F1-Macro)

Model	HALKWalker	NGramWalker	RandomWalker	WalkletWalker
XGBClassifier	0.9576	0.9168	0.9575	0.9281
LogisticRegression	0.9167	0.8461	0.9281	0.8864
RandomForest	0.9442	0.8924	0.947	0.9213
SVC	0.9398	0.8932	0.9486	0.9044
KNeighborsClassifier	0.9582	0.9010	0.9599	0.9178

6. Evaluation

We compared the effectiveness of using the graph embedding approach to the approach used in baseline models. The baseline involved ML models like Logistic Regression, Random Forest, MLP, RNN, SVC, and RNN with LSTM. However, these models didn't give the best results. For example, Random Forest achieved the highest accuracy of 44.7%, demonstrating moderate success of the baseline approach.

However, when integrating graph embedding techniques with walking strategies, it significantly reinforced our BiasLens's performance. As observed in Tables 1 through 3, our approach outperformed the baseline results, achieving approximately 84% accuracy on the low end and approximately 96% on the high end, which was for the combination of RandomWalker and KNeighborsClassifier. This highlights the superiority and significant benefits that came from incorporating a graph-based architecture with advanced walking strategies.

Table 5. BiasLens approach vs. Baseline

Model	Accuracy
Baseline Logistic Regression	0.383
BiasLens Logistic Regression	0.9296
Baseline RandomForest	0.447
BiasLens RandomForest	0.9478
Baseline SVC	0.402
BiasLens SVC	0.9449

7. Conclusion and Future Work

In this study, we presented a novel approach for detecting news bias using KGs. By constructing a KG from a news dataset obtained from AllSides.com and utilizing a walking strategy to extract graph embeddings for the articles, we trained an ML classifier model to predict news bias. Our approach achieved high accuracy, consistently surpassing 90%. Our results show great promise of utilizing KGs in bias detection tasks. This shows that there is a great value obtained from constructing a network and observing relationships made by articles, authors and outlets, which on its own achieved high accuracy scores.

Our Contributions lie within furthering the use of KG in conjunction with ML techniques, as news bias detection was not yet tackled in this way. We do however see that many more contribution can be made by building further atop our work, as we have not used the article texts to extract further information to create a complete feature vector for use in our classification. We do not hope to completely replace NLP based models for classification, rather we believe that there is also much to be extracted from article text, and with the advent of transformer based deep learning models, such as GPT [3] and Llama [15] which excel at language understanding, our approach may be used to enhance their performance by reducing hallucinations about real world events and keeping their responses grounded in the truth offered by the KG.

In the future we hope to dynamically update the KG with real-time news trends and evolving story coverage as major world events can be covered by a news outlet over a period of time which may be done by adding a version control system to keep track of graph changes periodically then reverting back to a previous version if needed. Adding to that we believe adding learnable weights to graph edges may be useful for providing explainability for the prediction results as it can be used to highlight the nodes and connections that have had the most impact on the models decision. There are much more features and contributions to be made to this project and we are excited to see what comes next.

References

- [1] Ramy Baly, Giovanni Da San Martino, James Glass, and Preslav Nakov. We can detect your bias: Predicting the political ideology of news articles, 2020.
- [2] Jess Joseph Benny. *Knowledge Informed Fake News Detection using Large Language Models*. PhD thesis, Univesity of Windsor, 2024.
- [3] Tom B. Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, Sandhini Agarwal, Ariel Herbert-Voss, Gretchen Krueger, Tom Henighan, Rewon Child, Aditya Ramesh, Daniel M. Ziegler, Jeffrey Wu, Clemens Winter, Christopher Hesse, Mark Chen, Eric Sigler, Mateusz Litwin, Scott Gray, Benjamin Chess, Jack Clark, Christopher Berner, Sam McCandlish, Alec Radford, Ilya Sutskever, and Dario Amodei. Language models are few-shot learners, 2020.
- [4] RDFLib Community. Rdfliib: A python library for working with rdf, 2023. Python Package Index.
- [5] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. Bert: Pre-training of deep bidirectional transformers for language understanding, 2019.
- [6] Erick Elejalde, Leo Ferres, and Eelco Herder. On the nature of real and perceived bias in the mainstream media. *PLOS ONE*, 13(3):1–28, 03 2018.
- [7] Linmei Hu, Tianchi Yang, Luhao Zhang, Wanjun Zhong, Duyu Tang, Chuan Shi, Nan Duan, and Ming Zhou. Compare to the knowledge: Graph neural fake news detection with external knowledge. In Chengqing Zong, Fei Xia, Wenjie Li, and Roberto Navigli, editors, *Proceedings of the 59th Annual Meeting of the Association for Computational Linguistics and the 11th International Joint Conference*

on *Natural Language Processing (Volume 1: Long Papers)*, pages 754–763, Online, August 2021. Association for Computational Linguistics.

- [8] Danyang Liu, Ting Bai, Jianxun Lian, Guangzhong Sun, Wayne Xin Zhao, Ji rong Wen, and Xing Xie. News graph: An enhanced knowledge graph for news recommendation. In *KaRS@CIKM*, 2019.
- [9] Yinhan Liu, Myle Ott, Naman Goyal, Jingfei Du, Mandar Joshi, Danqi Chen, Omer Levy, Mike Lewis, Luke Zettlemoyer, and Veselin Stoyanov. Roberta: A robustly optimized bert pretraining approach, 2019.
- [10] Iffat Maab, Edison Marrese-Taylor, and Yutaka Matsuo. Target-aware contextual political bias detection in news, 2023.
- [11] Mohit Mayank, Shakshi Sharma, and Rajesh Sharma. Deap-faked: Knowledge graph based approach for fake news detection, 2022.
- [12] Tomas Mikolov, Kai Chen, Greg Corrado, and Jeffrey Dean. Efficient estimation of word representations in vector space, 2013.
- [13] Petar Ristoski and Heiko Paulheim. Rdf2vec: Rdf graph embeddings for data mining. In Paul Groth, Elena Simperl, Alasdair Gray, Marta Sabou, Markus Krötzsch, Freddy Lecue, Fabian Flöck, and Yolanda Gil, editors, *The Semantic Web – ISWC 2016*, pages 498–514, Cham, 2016. Springer International Publishing.
- [14] Bram Steenwinckel, Gilles Vandewiele, Terencio Agozzino, and Femke Ongenaë. pyrdf2vec: A python implementation and extension of rdf2vec. In *European Semantic Web Conference*, pages 471–483. Springer Nature Switzerland, 2023.
- [15] Hugo Touvron, Thibaut Lavril, Gautier Izacard, Xavier Martinet, Marie-Anne Lachaux, Timothée Lacroix, Baptiste Rozière, Naman Goyal, Eric Hambro, Faisal Azhar, Aurelien Rodriguez, Armand Joulin, Edouard Grave, and Guillaume Lample. Llama: Open and efficient foundation language models, 2023.
- [16] Gilles Vandewiele, Bram Steenwinckel, Pieter Bonte, Michael Weyns, Heiko Paulheim, Petar Ristoski, Filip De Turck, and Femke Ongenaë. Walk extraction strategies for node embeddings with rdf2vec in knowledge graphs, 2020.